Trabalho 1 – Avaliação de Desempenho de uma Aplicação CRUD

Sistemas Distribuídos 1 | Aluno: _____ | Outubro de 2026

Parte 1 – Aplicação, Arquitetura e Cenários

Aplicação. Foi desenvolvido um CRUD de cadastro de pessoas em Python (Flask) com banco PostgreSQL. A tabela `pessoas` possui os campos id, nome, email, telefone, cidade, data_cadastro e status. Os testes usaram dois endpoints: `GET /api/pessoas` (leitura) e `POST /api/pessoas` (escrita).

Arquitetura. Cliente–servidor em três camadas: os clientes (simulados pelo Apache JMeter) enviam requisições HTTP ao servidor web Flask, que acessa o PostgreSQL via driver psycopg2.

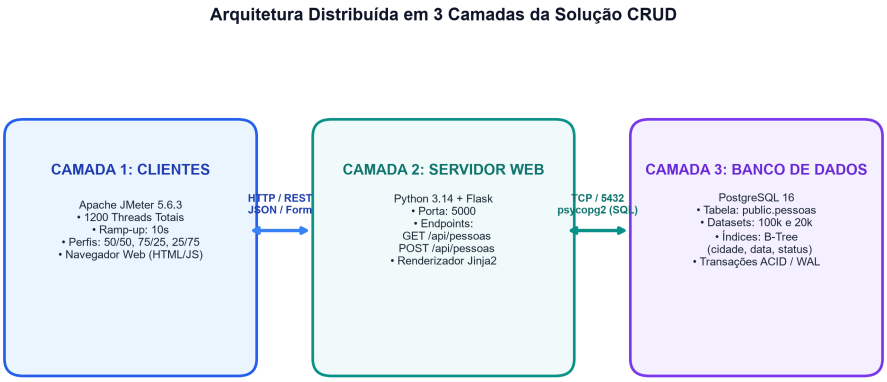


Figura 1 – Arquitetura da solução

Dataset. O banco foi populado com 100.000 registros gerados por um script Python (`pessoas.py`). Para a comparação pedida, também foi usada uma base reduzida de 20.000 registros.

Cenários. Cada teste usou 1.200 clientes simultâneos (ramp-up de 10 s), divididos conforme o enunciado:

- A) 50% leitura / 50% escrita – 600 clientes lendo e 600 escrevendo;
- B) 75% leitura / 25% escrita – 900 lendo e 300 escrevendo;
- C) 25% leitura / 75% escrita – 300 lendo e 900 escrevendo.

Os três cenários foram executados em três configurações de banco: 100k com índices, 20k com índices e 100k sem índices, totalizando 9 execuções.

Parte 2 – Resultados

Base	Cenário	Erro leitura	Erro escrita	Erro total	Tempo médio	Vazão
100k com índice	A (50/50)	10,00%	10,67%	10,33%	1120 ms	106 req/s
	B (75/25)	14,11%	14,67%	14,25%	1066 ms	110 req/s
	C (25/75)	6,67%	7,33%	7,17%	1089 ms	108 req/s
20k com índice	A (50/50)	7,33%	9,50%	8,42%	1057 ms	105 req/s
	B (75/25)	10,44%	10,00%	10,33%	1058 ms	108 req/s
	C (25/75)	4,00%	3,89%	3,92%	1013 ms	108 req/s
100k sem índice	A (50/50)	7,00%	6,83%	6,92%	1092 ms	105 req/s
	B (75/25)	10,00%	9,33%	9,83%	1060 ms	109 req/s
	C (25/75)	8,00%	6,44%	6,83%	1059 ms	108 req/s

Comparação entre cenários. A vazão ficou praticamente igual (≈105–110 req/s) em todas as execuções, e o tempo médio ficou em torno de 1 segundo. Isso indica que o limite está no servidor Flask, que atende as requisições de forma síncrona e abre uma conexão nova com o banco a cada requisição, e não no tipo de operação. A diferença entre os cenários apareceu principalmente na taxa de erros, que foi maior no cenário B (mais leituras) e menor no cenário C (mais escritas).

Respostas às Perguntas

1. O que acontece se o dataset for reduzido para 20k registros?

O desempenho melhorou. A taxa de erros caiu em todos os cenários (A: 10,33% → 8,42%; B: 14,25% → 10,33%; C: 7,17% → 3,92%) e o tempo médio diminuiu um pouco. Com menos dados, a tabela e os índices cabem melhor na memória do PostgreSQL, então há menos acesso a disco. A vazão não mudou, pois ela é limitada pelo servidor web.

2. O que acontece com a adição de índices? Qual escolher? A consulta melhora?

Foram criados índices B-Tree em `cidade`, `data_cadastro` e `status`. Nos testes, a versão com índices teve *mais* erros que a versão sem índices na maioria dos cenários (ex.: cenário A, 10,33% contra 6,92%). Isso acontece porque cada inserção precisa atualizar também os índices, deixando as escritas mais caras. Já as leituras não ganharam nada: a requisição de leitura do JMeter enviava o parâmetro `cidade`, que a API não utiliza, então o banco apenas listava os 100 registros mais recentes pela chave primária, sem usar os índices criados.

Portanto, a consulta não melhorou. O índice só traz benefício se a consulta realmente filtrar pela coluna indexada. Para buscar por cidade, o ideal seria a API filtrar com `WHERE cidade = ...` e manter o índice em `cidade`. Para a busca textual da aplicação (`ILIKE '%termo%'`), um índice B-Tree não funciona; o adequado seria um índice GIN com a extensão `pg_trgm`.

3. Como a escolha do banco (SQL ou NoSQL) impacta o sistema?

Um banco SQL como o PostgreSQL garante consistência e transações (ACID) e trabalha com esquema fixo, o que é adequado para um cadastro de pessoas. Em compensação, escala principalmente de forma vertical e o controle de transações gera custo sob muita concorrência. Um

banco NoSQL (MongoDB, Cassandra) costuma aceitar escritas com mais facilidade e escalar horizontalmente adicionando servidores, mas normalmente oferece consistência eventual e menos garantias. Para este sistema o PostgreSQL atende bem; um NoSQL só faria sentido com volume de escrita muito maior.

Conclusão

A aplicação suportou cerca de 107 requisições por segundo em todos os cenários, com taxa de erro entre 4% e 14%. O principal gargalo foi o servidor web síncrono, e não o banco. Reduzir a base para 20k diminuiu os erros, e os índices criados pioraram o desempenho porque aumentaram o custo das escritas sem serem usados nas leituras. Como melhorias, seria possível usar um servidor com vários workers (Gunicorn), um pool de conexões com o banco e índices adequados às consultas realmente feitas.